Chapter 1: Introduction to Parallel Processing

1.1 Parallel Computer Structures & Uniprocessor Parallelism

- Is there any form of parallelism in a uniproprocessor system? If yes, explain the different forms of parallelism used in a uniprocessor system and if no, explain why it is not feasible. *(2025 Fall)*

Yes. *Although a uniprocessor has only one CPU and is classified as* ***SISD****, it can still use internal parallelism to improve performance. The main forms are* ***Instruction-Level Parallelism (ILP), Data-Level Parallelism (DLP), and Thread-Level Parallelism (TLP).***

1. Instruction-Level Parallelism (ILP)

- ***Pipelining:*** *Divides instruction execution into stages such as Fetch, Decode, Execute, and Write-back.* ***It overlaps multiple instructions to increase throughput.***
- ***Superscalar Execution:*** *Uses multiple functional units to execute more than one independent instruction per cycle.* ***It increases processing speed.***
- ***Out-of-Order Execution:*** *Executes independent instructions before earlier stalled instructions.* ***It keeps the processor's functional units busy.***
- ***Branch Prediction & Speculative Execution:*** *Predicts the next branch and executes instructions before the result is known.* ***It reduces pipeline stalls.***
- ***Register Renaming:*** *Uses different physical registers to remove false dependencies.* ***It allows more instructions to execute in parallel.***

2. Data-Level Parallelism (DLP)

- ***SIMD/Vector Processing:*** *Performs the same operation on multiple data elements at once.* ***It speeds up data-intensive operations.***
- ***Parallel Arithmetic Hardware:*** *Uses fast arithmetic circuits such as carry-lookahead adders and fast multipliers.* ***It reduces computation time.***
- ***Memory Interleaving:*** *Divides memory into multiple modules for overlapping memory accesses.* ***It increases memory bandwidth.***

3. Thread-Level Parallelism (TLP)

- ***Hardware Multithreading/SMT:*** *Allows instructions from different threads to use processor resources concurrently.* ***It hides latency and improves processor utilization.***

Conclusion

Thus, a uniprocessor can exploit parallelism through ***ILP, DLP, and TLP****, even though it has only one CPU. These techniques improve performance by allowing independent instructions, data operations, or threads to* ***overlap or execute concurrently****.*

- What are the different mechanisms of parallelism used in a uniprocessor system? Explain. *(2024 Fall)*
- What is parallelism in a computer system? Explain different techniques used to enhance the performance in a uniprocessor system? Why is the importance of parallelism increasing day by day? Explain with suitable examples. *(2025 Fall Mock)*

1.2 Architectural Classification & Applications

- What is parallel processing? Why is the demand of parallel processing increasing day by day? Explain with examples. *(2025 Fall)*
- Flynn's classification *(Short Note, 2025 Fall)*
- Explain the architectural classification of computer systems based on Flynn's taxonomy. Provide examples for each category and discuss their relevance in parallel computing. *(CA / Old)*

***Flynn's Taxonomy**, proposed by **Michael J. Flynn in 1966**, classifies computer architectures based on the number of **instruction streams** and **data streams** they process.*

- ***Instruction stream:** Sequence of instructions executed by the processor.*
- ***Data stream:** Data on which the instructions operate.*

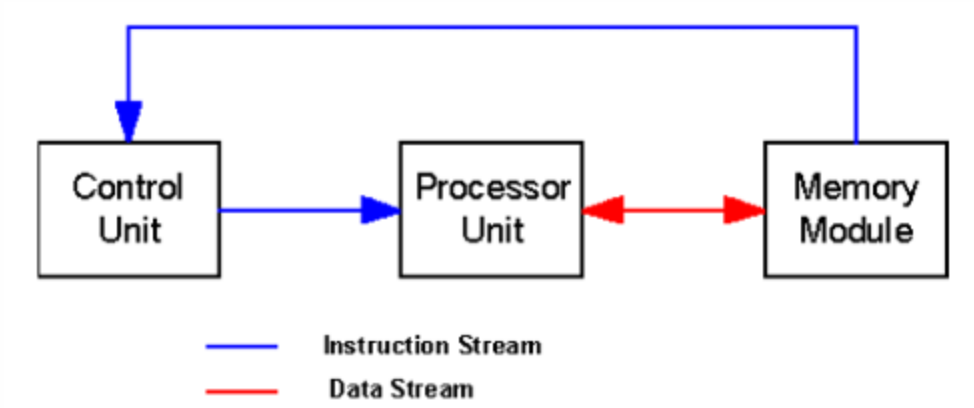
*It has **four categories: SISD, SIMD, MISD, and MIMD.***

---

1. SISD — Single Instruction, Single Data

What it is:
*SISD is a **sequential architecture** in which one processor executes one instruction stream on one data stream.*

How it works:
The processor fetches and executes instructions one after another.



Example:
Intel 8085 and traditional single-core uniprocessor systems.

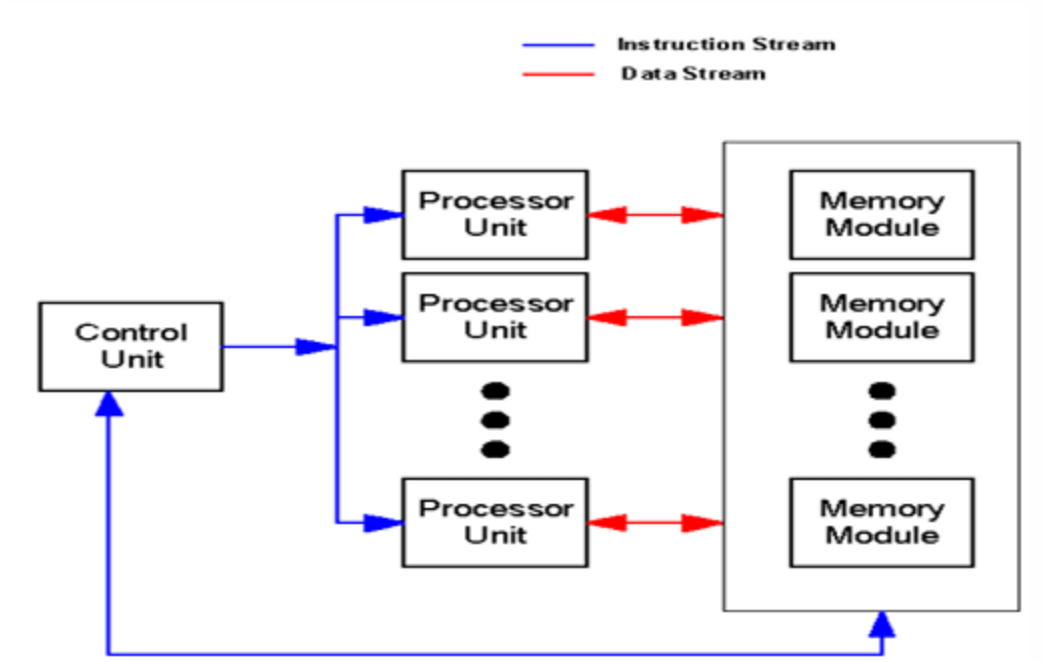
Relevance:
It provides little explicit parallelism, but modern SISD processors can still exploit ***instruction-level parallelism (ILP)*** *using techniques such as pipelining and superscalar execution.*

---

2. SIMD — Single Instruction, Multiple Data

What it is:
SIMD executes ***one instruction on multiple data elements simultaneously****.*

How it works:
A single instruction is applied to different data items at the same time, allowing the same operation to be performed in parallel.



Example:
*Cray vector processors, CPU vector instructions such as **SSE/AVX**, and GPUs.*

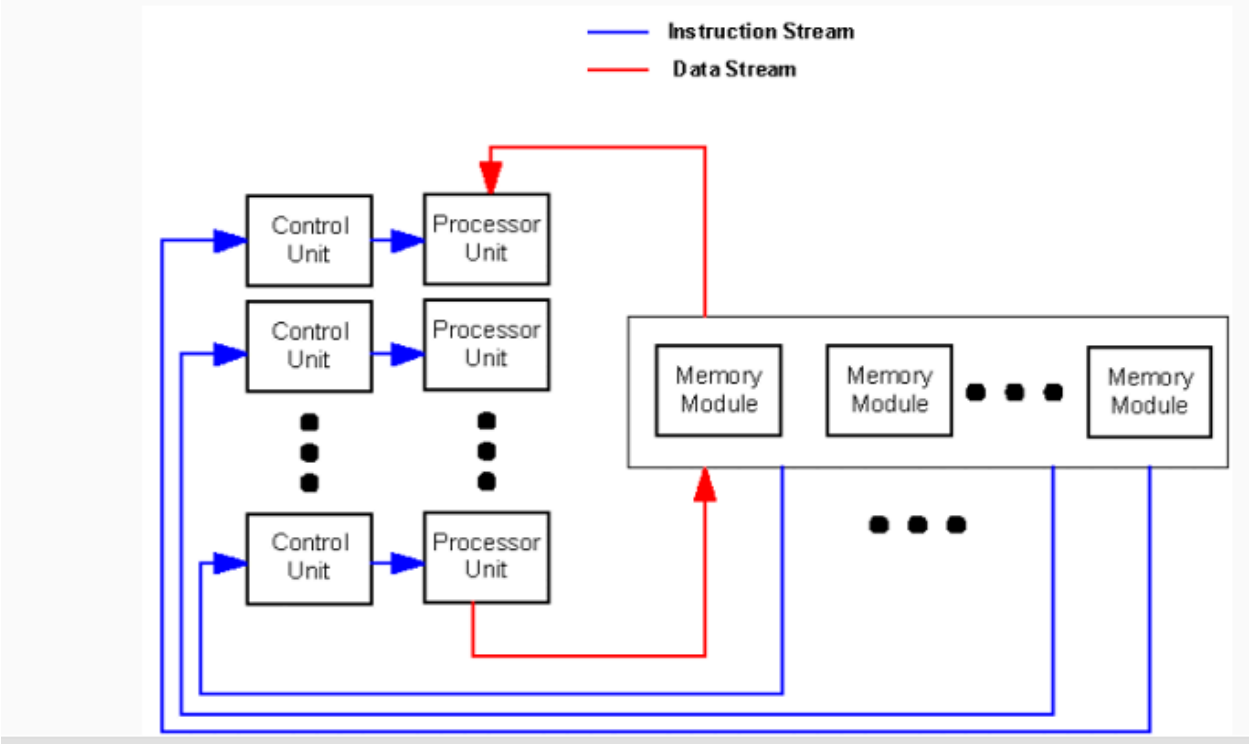
Relevance:
*SIMD is mainly used for **data-level parallelism (DLP)**. It is effective for image processing, matrix operations, scientific computation, graphics, and machine learning.*

---

3. MISD — Multiple Instruction, Single Data

What it is:
*MISD uses **multiple instruction streams operating on the same data stream**.*

How it works:
Different processing units perform different operations on the same input data.



Example:
*It is mainly found in **specialized fault-tolerant and safety-critical systems**, such as aerospace or control systems.*

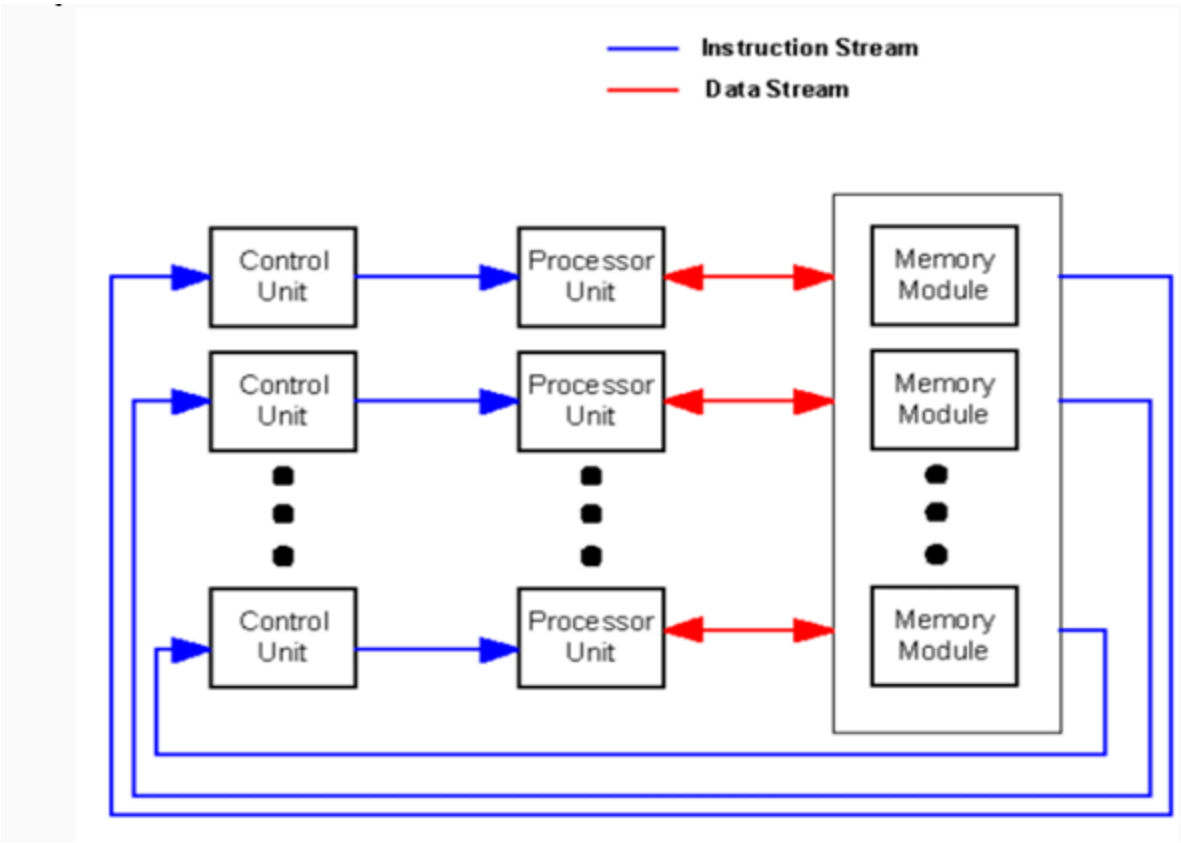
Relevance:
*MISD is **rare in general-purpose computing**. It is mainly useful where reliability, redundancy, or fault detection is important.*

---

4. MIMD — Multiple Instruction, Multiple Data

What it is:
*MIMD uses **multiple processors or processing elements**, where each can execute a different instruction stream on different data.*

How it works:
Processors operate independently and can execute different tasks simultaneously.



Examples:
Multicore CPUs such as Intel Core and AMD Ryzen, HPC clusters, and distributed computing systems.

Relevance:
*MIMD is the **most widely used architecture for general-purpose parallel computing**. It supports **task-level parallelism (TLP)** and can handle different tasks and large workloads simultaneously.*

Main types:

- ***Shared-memory:** Processors share a common memory.*
- ***Distributed-memory:** Each processor has its own local memory and communicates through a network.*

Comparison

Classification	***Instruction Streams***	***Data Streams***	***Main Parallelism***	***Example***

SISD	*1*	*1*	*Limited/ILP*	*Intel 8085*
SIMD	*1*	*Multiple*	*Data-level*	*GPU, AVX*
MISD	*Multiple*	*1*	*Specialized/redundancy*	*Fault-tolerant systems*
MIMD	*Multiple*	*Multiple*	*Task-level*	*Multicore CPU, HPC cluster*

Conclusion

*Flynn's Taxonomy provides a simple way to understand computer architectures according to their **instruction and data streams**. **SISD** is mainly sequential, **SIMD** exploits data-level parallelism, **MISD** is mainly used in specialized applications, and **MIMD** provides flexible and scalable parallel processing and is therefore dominant in modern parallel computing.*

- Compare SIMD and MIMD architectures in terms of design and applications. *(2025 Fall New)*
- Explain how parallel processing improves performance compared to sequential processing with appropriate examples. *(2025 Fall New)*

1.3 Need of parallel programming, characteristics of parallel algorithms

- What is the need of parallel programming? Explain the required characteristics of parallel algorithms. *(2025 Fall)*

Need for Parallel Programming

*Parallel programming is needed to **solve large and complex problems efficiently by using multiple processors or cores simultaneously**.*

- ***Performance Improvement:** It splits a problem into smaller tasks and distributes them among multiple processors, **allowing** faster execution.*
- ***Scalability for Large Data Sets:** It distributes large workloads across multiple processing resources, **allowing** large amounts of data to be processed efficiently.*
- ***Real-Time Processing:** It processes tasks concurrently, **ensuring** timely results and faster decision-making.*

- ***Efficient Resource Utilization:*** *It uses available CPU cores, memory, and other hardware resources effectively,* ***increasing*** *overall resource utilization.*
- ***Enabling Advanced Technologies:*** *It is mostly used in* ***AI and machine learning, GPU computing, graphics rendering, and scientific simulations****,* ***allowing*** *complex computations to be performed efficiently.*

Required Characteristics of Parallel Algorithms

A good parallel algorithm should have the following characteristics:

1. Decomposition

It divides a large problem into smaller tasks.

It helps *distribute work among processors,* ***allowing*** *tasks to execute in parallel.*

2. Concurrency

It provides multiple independent tasks that can execute simultaneously.

It helps *keep processors busy,* ***allowing*** *more work to be completed at the same time.*

3. Load Balancing

It distributes the workload evenly among processors.

It helps *prevent some processors from being overloaded while others remain idle,* ***ensuring*** *better resource utilization.*

4. Low Communication

It minimizes data exchange between processors.

It helps *reduce communication delays,* ***allowing*** *more time for useful computation.*

5. Synchronization

It coordinates the execution of parallel tasks when required.

It helps *prevent race conditions and inconsistent results,* ***ensuring*** *correct execution.*

6. Scalability

It should perform efficiently when the number of processors or problem size increases.

It helps *the algorithm take advantage of additional processing resources,* ***allowing*** *performance to scale.*

7. Low Overhead

It minimizes the extra cost of task creation, communication, and synchronization.

It helps *ensure that the performance gain from parallelism is greater than the overhead.*

8. High Speedup and Efficiency

A good parallel algorithm should achieve high ***speedup*** *and* ***processor efficiency****.*

- ***Speedup:*** $(S = T_s/T_p)$
- ***Efficiency:*** $(E = S/p)$

They help *measure how effectively multiple processors are being utilized.*

Conclusion

Thus, parallel programming is mainly used to ***improve performance, process large workloads, and utilize multiple hardware resources effectively****. A good parallel algorithm should provide* ***decomposition, concurrency, load balancing, low communication, synchronization, scalability, low overhead, and high speedup and efficiency****,* ***ensuring*** *effective use of parallel processing resources.*

1.4 Parallel Programming Techniques

- Explicit vs. implicit parallel programming *(Differentiate, 2025 Fall)*

In parallel computing, developers and systems use two primary approaches to expose and exploit parallelism: ***implicit*** *and* ***explicit parallel programming****. While implicit parallel programming relies on the compiler and operating system to automatically uncover and manage concurrency, explicit parallel programming places the responsibility of parallel execution design directly on the programmer.*

Implicit vs. Explicit Parallel Programming

Feature	*Implicit Parallel Programming*	*Explicit Parallel Programming*
Primary Responsibility	*The compiler and operating system.*	*The programmer.*
Source Language	*Conventional sequential languages (e.g., standard C, C++, Fortran, Pascal).*	*Concurrent programming languages (e.g., Chapel, Occam) or sequential languages extended with parallel libraries/directives (e.g., OpenMP, MPI).*
Compiler Role	***Parallelizing / Vectorizing Compiler:*** *detects independent operations, and automatically generates parallel object code.*	***Concurrency-Preserving Compiler:*** *Preserves programmer-defined parallelism and maps it onto machine resources.*
Programmer Effort	*Low; the programmer does not need to understand the underlying parallel architecture or parallel semantics.*	*High;designs parallel algorithms and manages synchronization and communication.*
Resource Allocation	*Handled by OS, hardware, or compiler.*	*Guided explicitly by programmer directives or library parameters.*
Examples	*Multi-threaded applications using parallelizing compilers, loop vectorization.*	*MPI (Message Passing Interface), OpenMP, PThreads, CUDA, and Chapel programming language.*

- Explicit and implicit parallel programming *(Short Note, 2025 Fall Mock & 2025 Fall New)*

1.5 Parallel Programming Models

- What is the message passing programming? In which case is it suitable? Explain. *(2025 Fall)*
- In which architecture is the message passing programming model used? Compare the message passing programming model with the shared memory programming model. *(2024 Fall)*

The message passing programming model is used in distributed memory parallel architectures. These systems are also referred to as loosely coupled architectures or multicomputers, where there is no physical sharing of memory between processing nodes. Instead, each node operates as a complete computer building block, containing its own private processor, local memory, and I/O system

Feature	Shared Memory Model (e.g., OpenMP)	Message Passing Model (e.g., MPI)
Target Architecture	Shared memory systems (SMP)	Distributed memory systems (Clusters, MPPs)
Address Space	Unified, global address space	Divided, private local address spaces
Communication	Implicit (direct memory read/write)	Explicit (`send`/`receive` library calls)
Processes/Threads	Thread-based execution (via `fork`/`join`)	Process-based execution
Data Partitioning	Managed automatically in main memory	Programmed explicitly by the developer
Synchronization	Manual locks, semaphores, and barriers	Built into synchronous message transfers

Scalability	Limited by bus/memory bottlenecks	High; scales to thousands of nodes
Complexity	Low to moderate; simple annotations	High; complex manual coordination

The message passing model is highly suitable and preferred in the following scenarios:

1. ***Distributed-Memory Architectures:***
   *Suitable for **PC clusters, distributed-memory multicomputers, and supercomputers** where processors do not share common main memory.*
2. ***Massively Scalable Applications:***
   *Suitable for applications requiring **thousands or millions of parallel processes**, especially in **HPC and big data analytics**.*
3. ***Domain Decomposition:***
   *Suitable for **data-intensive problems** where data is divided into subsets, such as **weather forecasting, matrix multiplication, and fluid dynamics**.*
4. ***Hybrid Implementations:***
   *Suitable for **multi-core, multi-node clusters**, where **MPI** handles inter-node communication and **OpenMP** handles intra-node execution.*

1.6 Parallel Algorithms for Multiprocessors

- Synchronized vs. asynchronized parallel algorithms *(Differentiate, 2025 Fall)*
- Explain the Synchronized and Asynchronized Parallel Algorithms for Multiprocessors. *(2025 Fall Mock)*

Synchronized vs. Asynchronized Parallel Algorithms

*Parallel algorithms are broadly classified into **Synchronized (Synchronous)** and **Asynchronized (Asynchronous) Parallel Algorithms** based on how processors coordinate their work and manage data dependencies.*

1. Synchronized Parallel Algorithms

*In a **synchronized parallel algorithm**, processors coordinate their progress at predefined **synchronization barriers**.*

- ***Step-by-Step Execution:*** *Processors complete step (i) before proceeding to step (i+1).*
- ***Strict Determinism:*** *Provides predictable data access and simpler design and debugging.*
- ***Communication:*** *Uses shared variables, locks, semaphores, or tightly coupled networks.*
- ***Example: Parallel Binary Summation*** *and* ***Cannon's Algorithm****.*
- ***Pros:*** *Predictable, deterministic, and avoids race conditions.*
- ***Cons:*** *Synchronization overhead and* ***load imbalance****; faster processors may wait for slower processors.*

2. Asynchronized Parallel Algorithms

In an ***asynchronized parallel algorithm****, processors operate independently without global execution barriers.*

- ***Independent Task Progress:*** *Each processor executes tasks at its own pace.*
- ***Latency Hiding:*** *Execution continues during slow memory or network operations.*
- ***Asynchronous Communication:*** *Uses buffered, non-blocking message-passing.*
- ***Examples: Pure Dataflow Computing, non-blocking MPI, and Dynamic Work-Stealing****.*
- ***Pros:*** *Maximizes hardware utilization, throughput, and scalability.*
- ***Cons:*** *Complex to design and susceptible to* ***race conditions*** *and* ***deadlocks****.*

1.7 Performance of Parallel Algorithms

- In the example of adding 'n' numbers using 'p' processing elements, can granularity be achieved in a cost-optimal manner? Provide justification for your answer. **OR** Is it possible to implicitly reduce the size of an instruction? Explain your reasoning. *(CA / Old)*